Mock Exam B — Weeks 1 to 7: Answers

DSA27 · Data Structures and Algorithms · model answers and marking points

Question bank practice paper

Fall 2026

Contents

Section A — Multiple choice **1**

Section B — Written answers **4**

> *Sit the paper first. Each answer gives the question's number in the bank (for example W4-M23), so you can find the lecture it comes from and more questions like it.*

Section A — Multiple choice

#	Answer	Bank	#	Answer	Bank	#	Answer	Bank
A1	**b**	W1-M28	A11	**b**	W3-M41	A21	**b**	W6-M23
A2	**c**	W1-M33	A12	**c**	W4-M23	A22	**c**	W6-M27
A3	**d**	W1-M36	A13	**a**	W4-M24	A23	**d**	W6-M31
A4	**a**	W2-M29	A14	**b**	W4-M30	A24	**b**	W6-M37
A5	**b**	W2-M31	A15	**d**	W4-M36	A25	**d**	W6-M38
A6	**c**	W2-M34	A16	**a**	W5-M25	A26	**a**	W7-M24
A7	**d**	W2-M35	A17	**c**	W5-M27	A27	**d**	W7-M25
A8	**d**	W3-M28	A18	**a**	W5-M29	A28	**b**	W7-M30
A9	**a**	W3-M31	A19	**d**	W5-M36	A29	**b**	W7-M36
A10	**c**	W3-M34	A20	**c**	W5-M37	A30	**a**	W7-M39

Lecture 1 — Python and the course

A1 — b. *(W1-M28)* `:03d` means an integer, at least 3 wide, padded with zeros: `007`. `:.2f` means two digits after the point: `3.14`. (a) ignores the format, and (d) has the wrong precision.

A2 — c. *(W1-M33)* $O(n^2)$: ten times the data means $10^2 = 100$ times the work, so about 200 seconds. (a) is what an $O(n)$ program would do.

A3 — d. *(W1-M36)* Most decimal fractions are not exact in binary (`0.1 + 0.1 + 0.1 == 0.3` is `False`), and the errors add up over many sums. Store the smallest unit as an `int`, which is exact. Floats can be negative (b) and go far above a million (c).

Lecture 2 — Complexity and arrays

A4 — a. *(W2-M29)* $1000n < n^2$ exactly when $n > 1000$. Big-O describes growth for large n, so (b) is wrong for small n; (c) ignores that n^2 overtakes any $1000n$; being polynomials (d) says nothing about which is smaller.

A5 — b. *(W2-M31)* The loop stops when $i^2 \geq n$, that is after about $\sqrt{n}$ steps (n = 1,000,000 gives i = 1,000). It neither halves (a) nor walks all the way to n (c).

A6 — c. *(W2-M34)* After the shift, slot `size - 1` still holds a copy of the last reference. Clearing it means nothing reads a stale value, and the object can be freed. `len(arr)` is the fixed capacity and never changes (a); the order (b) was already right after the shift.

A7 — d. *(W2-M35)* C does not check indices: `a[6]` is base + 6 × 4, the memory right after the array. No error is raised (a), and an array cannot grow (b) or wrap (c). The course `Array` checks, and raises `IndexError`.

Lecture 3 — Recursion

A8 — d. *(W3-M28)* gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0), which returns 6: four calls. (b) forgets the last call, the one that reaches the base case b == 0; (c) confuses the number of calls with the size of b. Euclid's algorithm is O(log min(a, b)), so the count stays small.

A9 — a. *(W3-M31)* The exponents are 1000, 500, 250, 125, 62, 31, 15, 7, 3, 1 and 0: one frame each, 11 at the deepest point, about $\log_2 n$. (b) and (d) describe an $O(n)$-deep power; (c) ignores the frames that are waiting for their recursive call to return.

A10 — c. *(W3-M34)* With <, a tie takes the head of `right` first, so two equal keys swap their original order: the merge is still correct but not **stable**, and the test with `(1, "L")` and `(1, "R")` catches it. No value is lost (a), because one of the two heads is always taken; the cost is unchanged (b); equal values compare without error (d).

A11 — b. *(W3-M41)* `subsets([2])` is `rest + [[2] + s for s in rest]` with `rest = [[]]`, so `[[], [2]]`. Then `subsets([1, 2])` is `[[], [2]]` (the subsets without 1) followed by `[[1], [1, 2]]` (the same with 1 in front). (a) is the same subsets in another order, which the tests would also accept after sorting, but it is not what this code returns.

Lecture 4 — Dynamic arrays

A12 — c. *(W4-M23)* Resizes on appends 2, 3 and 5 take the capacity 1 → 2 → 4 → 8; the sixth append fits. `len` is the size, 6. (a) confuses size with capacity; (d) grows by a factor of 3 from 4.

A13 — a. *(W4-M24)* The `Array` has all `capacity` slots (so (b) is wrong), but the ones beyond `size` are not part of the user's data. Reading one must be an `IndexError`, not a silent `None` — W4-B2 is exactly this bug.

A14 — b. *(W4-M30)* The course `DynamicArray.pop` shifts and clears a slot, but never resizes: the capacity stays 16. Shrinking at one quarter (c) is the Homework 4 extension, `ShrinkingArray` (W4-C3), not the course class.

A15 — d. *(W4-M36)* Constant-step copying is quadratic, about $n^2/(2 \times 10)$: double n and the copies multiply by about 4 (3,196,800 in fact). (b) is what doubling gives — linear.

Lecture 5 — Linked lists

A16 — a. *(W5-M25)* Reading allows `0 <= index < len`, so `ll[3]` is out of range. Inserting allows `0 <= index <= len`, so (b) is a legal append at the end and (d) is `push_front`. `ll[-1]` (c) is 3: `__getitem__` adds `_size` to a negative index.

A17 — c. *(W5-M27)* The node before x must point past it, and the node after x must point back past it. (a) and (b) change only x's own references, which leaves the neighbours still linked to x. (d) makes each neighbour point at itself.

A18 — a. *(W5-M29)* After `pop_front`, `head` is `None` but `_tail` still refers to the old node 7. `append` sees a tail that is not `None`, so it links 5 after that dead node and never sets `head`. The walk from `head` finds nothing, while `_size` went from 0 to 1. It fails silently and raises no error (d). Every method that can empty the list must clear `_tail`.

A19 — d. *(W5-M36)* The contract raises `IndexError("pop from empty list")`, like `list.pop()`. Without the check, `self.head.value` would raise `AttributeError` (c) on `None`, which is W5-B3.

A20 — c. *(W5-M37)* Inserting before a node means changing its **predecessor's** `next`, and a singly linked node has no reference back. Inserting **after** it is $O(1)$ (a). A doubly linked list makes both $O(1)$.

Lecture 6 — Stacks

A21 — b. *(W6-M23)* * is stacked first; when + arrives, * has higher precedence and leaves first, so `2 3 *` comes before `4 +`. (a) is `2 * (3 + 4)`, and (c) and (d) are not postfix.

A22 — c. *(W6-M27)* `6 / 2` is 3.0 (true division gives a float), then `3.0 * 3` is 9.0. (a) multiplies first, which the postfix order does not allow; exactly one value is left at the end, so there is no `ValueError` (d).

A23 — d. *(W6-M31)* `((a)` leaves one `(` on the stack, which only the empty-at-the-end check catches. The partner check still catches (a), and the non-empty check still catches (b) and (c), whose closer arrives on an empty stack.

A24 — b. *(W6-M37)* `Stack(values)` pushes the values in order, so the last one, 3, is the top. A stack built from values is not empty (d).

A25 — d. *(W6-M38)* A peek that removes is a pop: the second call sees the item underneath. Reading index 0 (a) is wrong too, but it would return the **same** wrong value twice.

Lecture 7 — Queues

A26 — a. *(W7-M24)* The three dequeues leave D, E in slots 3 and 4, with head 3. F goes to `(3 + 2) % 5 = 0`, G to slot 1, and slot 2 is still empty. (b) is `list(q)`, the queue order, not the block. (d) forgets that dequeue clears the slot it read, so C is gone. (c) has seven slots in a five-slot block.

A27 — d. *(W7-M25)* `_grow` copies in queue order: C D E F go to slots 0–3 and head becomes 0. Then G is written at `(0 + 4) % 8 = 4`. (a) is what happens if head is kept, which is the naive grow of W7-M09. (b) would overwrite the front.

A28 — b. *(W7-M30)* One slot always stays empty, so that `head == tail` can only mean empty. Eight slots hold seven items. (a) would need a size field, which is what `dsa/queue.py` keeps.

A29 — b. *(W7-M36)* The stack hands back the most recently found node first, so the search goes as deep as it can before backing up. That is why Week 14 puts `bfs` and `dfs_iterative` side by side: the loops are the same, and only the structure differs. (c) cannot happen as long as visited nodes are not added again.

A30 — a. *(W7-M39)* Slot of offset i is `(4 + i) % 6`, for i = 0 to 4: 4, 5, then round to 0, 1, 2. (b) forgets the %; (c) ignores head; (d) walks backwards.

Section B — Written answers

Award a mark for each marking point your answer makes clearly and correctly — not for length, and not for points the question did not ask for. Half marks are fine where a point is half made.

#	Bank	Lecture	Marks
B1	W1-E8	1 — Python and the course	3
B2	W2-E7	2 — Complexity and arrays	4
B3	W3-E8	3 — Recursion	4
B4	W4-E6	4 — Dynamic arrays	4
B5	W5-E7	5 — Linked lists	4
B6	W6-E6	6 — Stacks	4
B7	W6-E7	6 — Stacks	3
B8	W7-E6	7 — Queues	4
		Total	**30**

B1 *(3 marks — W1-E8)*

- `def f(x): print(x * 2)` shows the value on the screen and returns `None`; `def g(x): return x * 2` hands the value back to the caller.
- A test sees only what is **returned**: `assert f(3) == 6` fails, because `f(3)` is `None`, although "6" appeared on the screen.
- Exercises take **parameters** and **return** results because a test can call them with any input and check the output automatically. A function that calls `input()` waits for a keyboard that a test does not have, and `print()` gives the test nothing to check.

B2 *(4 marks — W2-E7)*

- **(1)** One pass over n items with an O(1) body: **Rule 2**, $\Theta(n)$.
- **(2)** i is halved until it reaches 1: **Rule 4**, $\Theta(\log n)$.
- **(3)** n iterations (**Rule 2**); the body hides a slice and an `in` (**Rule 5**), but the slice copies at most **10** items and `in` searches at most 10 — a constant, not n. So the body is O(1) and the loop is $\Theta(n)$. A hidden loop costs what it actually walks: here a fixed 10, not the whole list.

- **Total**, by **Rule 1**: $\Theta(n + \log n + n) = \Theta(n)$. (Timing it confirms this: doubling n doubles the time.)

B3 *(4 marks — W3-E8)*

- (i) $T(n) = T(n-1) + c \to O(n)$ calls and multiplications, and n + 1 frames deep: `power(2, 4096)` raises `RecursionError`.
- (ii) $T(n) = T(n/2) + c \to O(\log n)$: for n = 4,096 it makes 14 calls (exponents 4096, 2048, ..., 1, 0) and 14 multiplications, 14 frames deep.
- (iii) $T(n) = 2T(n/2) + c \to O(n)$: two calls on half the input undo the halving. For n = 4,096 it makes 16,383 calls. It is only about 14 deep, so it does not crash, but it is no faster than (i).
- The lesson: count the recursive calls *and* their size. Calling once and keeping the result in a variable is what makes (ii) logarithmic.

B4 *(4 marks — W4-E6)*

- **Bounds:** `0 <= 1 <= 4` holds. **Full:** size 4 = capacity 4, so resize to 8: allocate an `Array(8)` and copy `a b c d` — **4 copies**.
- **Shift right**, from the end down to the gap: `d` to slot 4, `c` to 3, `b` to 2 — **3 shifts**. Then write `"x"` into slot 1; size becomes 5.
- **Result:** `a x b c d`, size 5, capacity 8; 7 elements moved, plus 1 write.
- **Why first:** the first shift writes into slot `size`, which does not exist in a full block, so shifting before resizing raises `IndexError`.

B5 *(4 marks — W5-E7)*

- **Steps:** raise `IndexError` unless `0 <= index <= len` (`index == len` means "at the end"). If `index == 0`, call `push_front`, because there is no node before the gap. Otherwise walk `prev` from the head `index - 1` steps, then `prev.next = Node(value, prev.next)` and `_size += 1`.
- **Why `index - 1`:** only the node **before** the gap holds the reference that must change. It is at index `index - 1`, which is `index - 1` steps from the head.
- **Cost:** best case $O(1)$, at index 0 or 1. Worst case $O(n)$, at the end. The splice itself is always $O(1)$.
- **`range(index)`:** the walk goes one node too far. `insert_at(1, 9)` on `[1, 2, 3]` gives `[1, 2, 9, 3]`, with no error. At `index == len`, `prev` walks off the end to `None`, and `prev.next` raises `AttributeError`.

B6 *(4 marks — W6-E6)*

- **Still right:** `3 + 4 * 2`. It gives `3 4 2 * +` = 11 under both rules: when * arrives, + is lower, and the two rules differ only for **equal** precedence.
- **Wrong:** `8 - 3 - 2`. The second - does not pop the first, so the output is `8 3 2 - -`, which is `8 - (3 - 2)` = 7 instead of `8 3 - 2 -` = 3.
- **Why tests miss it:** only an input with two operators of equal precedence **in a row**, where the operation does not associate (-, /), shows the difference. `2 + 3 + 4` gives a different postfix but the same value, so a test that checks only values passes.
- **`>` is right** for a right-associative operator such as **: `2 ** 3 ** 2` means `2 ** (3 ** 2)`, so the later ** must wait on top of the earlier one.

B7 *(3 marks — W6-E7)*

- **Postfix:** `8 2 - 1 2 + / 4 *`. The `)`s output `-` and `+`; when `*` arrives, the `/` on the stack has equal precedence and leaves first.
- **Evaluation** (stack bottom → top): `-` pops 2 and 8 → [6]; `+` pops 2 and 1 → [6, 3]; `/` pops 3 and 6 → [2.0]; `*` pops 4 and 2.0 → [8.0]. Exactly one value is left: the answer, 8.0.
- **A float** because `/` is true division in Python: `6 / 3` is 2.0, and every result built on it stays a float.

B8 *(4 marks — W7-E6)*

- **No `%` on the tail:** once the back reaches the end of the block, the write goes to slot `capacity` and the `Array` raises `IndexError`, although there are free slots at the front. Shortest: `CircularQueue(2)`: enqueue, enqueue, dequeue, enqueue (tail = 1 + 1 = 2).
- **No `%` on the head:** dequeues work until head reaches `capacity`; the next dequeue reads slot `capacity` and raises. Shortest: `CircularQueue(1)`: enqueue, dequeue (head = 1), enqueue (written to `(1 + 0) % 1 = 0`), dequeue.
- **`_capacity` not updated in `_grow`:** the new block is bigger, but every `%` still wraps at the old size, so the next enqueue after the grow overwrites the front, silently. Shortest: fill a ring of capacity 1, enqueue twice more.
- **Why the tests miss them:** while the ring has not wrapped, head is 0 and `head + size` is below the capacity, so `%` changes nothing. Every test must take the ring **round** at least once, and grow it after it has wrapped.